KVM MC • LPC 2026

Orphaned Virtual Machines

Preserving vCPU Execution Across Kexec

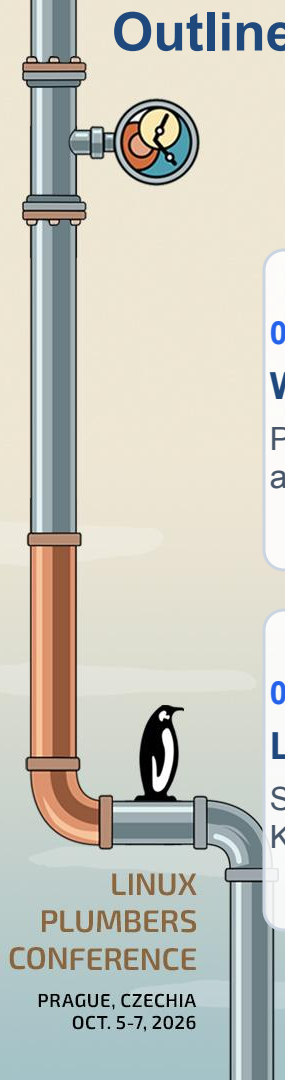
The Caretaker Architecture for Zero-Blackout Live Update

Pasha Tatashin
Google



LINUX
PLUMBERS
CONFERENCE
PRAGUE, CZECHIA / OCT. 5-7, 2026

Outline



01
What is Live Update
Preserving key hardware state across kernel reboots

02
VM Suspend
Understanding CPU execution gaps in live update flows

03
Orphan VM Live Demo
Live demonstration of zero-blackout guest handover

04
Live Update Stack
Subsystem decomposition from Kexec to KVM

05
Open Questions
Discussion & open upstream Plumbers topics

06
Resources & Community
Active review series, OVM design doc & bi-weekly sync

What is Live Update?

Preserving key hardware state across host kernel reboots

CORE CAPABILITY

Live Update enables updating the full host software stack across a reboot into a new kernel image while keeping resources alive.

Preserved Resources

- **Guest Memory:** memfd / guest_memfd
- **Pass-Through Devices:** VFIO/PCI/IOMMU state
- **Infrastructure:** KHO & LUO kernel subsystems

KEY DRIVERS & VALUE

- **Eliminates Live Migration:** Avoids network memory copies and spare host capacity overheads.
- **Accelerator Support:** Non-destructive updates for GPUs & TPUs where downtime is critical.
- **Rapid Security Patching:** Rapid deployment of complex fixes (data structures, assembly) unsupported by live patching.
- **LLM Engineering Era:** Fast vulnerability fixes deployed without stopping VMs.

The Problem: VM Suspend

Traditional Live Update preserves memory & devices, but stalls CPU execution

STANDARD LIVE UPDATE FLOW

1. Suspend: Guest vCPUs are paused and state is serialized to userspace.

2. Reboot: Outgoing kernel reboots into incoming kernel via **kexec**.

3. Resume: New VMM spawns and restores vCPU state.

Result: Stalls guest vCPU execution for several seconds, causing a service blackout.

IMPACT OF THE BLACKOUT GAP

- **In-Flight Operations:** While DMA and PCI devices stay active, vCPUs are suspended.
- **Strict SLAs:** AI inference and training workloads experience noticeable latency spikes.
- **LLM Engineering Era:** Rapid security patches require frequent updates, making repeated multi-second gaps unacceptable.

Orphaned VM Demo



Live Update Stack

New Orphan VM Stack	KVM Caretaker	**CONFIG_KVM_CARETAKER Hypervisor of Orphaned VMs (kvm/caretaker/)** Wraps preserved vCPUs as oncore_job instances to run guest entry/exit loops
	On-Core Scheduler	**CONFIG_LIVEUPDATE_ONCORE Sched. for Orphaned Jobs (.../oncore.c)** Workload-agnostic job abstraction; time-sliced round-robin FIFO runqueue
	Physical CPU LUO Preservation	**CONFIG_LIVEUPDATE_CPU, Physical CPU Preservation (.../cpu_preserve.c)** Parks CPUs in isolated .cpu_preserved.* sections; maintains page tables
	vCPUfd LUO Preservation	**KVM_CAP_VCPU_PRESERVE vcpufd & vmfd (kvm_luo.c)** Preserves vcpufd, per-cpu data, vmfd, and secondary page tables across kexec via LUO.
Mainline Linux	Live Update Orchestrator	**LUO** Provides uAPI, Resource ownership, State Machine, and Session Management.
	Kernel HandOver	**KHO** Preserve Kernel Memory and cross Kernel ABI / Versioning
	WARM REBOOT	**KEXEC** Skips firmware/hardware re-initialization; current Linux kernel acts as boot loader for the next Linux kernel.

KVM Caretaker State Machine & Re-Attachment Handshake

Cross-Kernel Synchronization (cb->state & kvm_caretaker_wait_for_attach())

1. OUTGOING DETACH

State: PAUSED

- Outgoing KVM serializes baseline vCPU registers into `@arch_state`.
- Initializes `cb->state = KVM_CARETAKER_PAUSED`.
- Cleans cache lines to PoC (`cpu_preserved_clean()`).
- Activates the `oncore_job`.

2. EXECUTION LOOP

PAUSED ↔RUNNING

Quantum Entry:

Preserved pCPU transitions **PAUSED → RUNNING**, loads `@arch_state` into hardware (VMCS / VMCB / EL2), and enters guest mode.

Quantum Exit:

On timer expiry, HLT/WFI yield, or unhandled exit (STALL), Caretaker saves live registers back to `@arch_state` and transitions **RUNNING → PAUSED**.

3. RE-ATTACHMENT

`kvm_caretaker_wait_for_attach()`

Fast Path (0 ns wait):

If `cb->state == PAUSED`, `cmpxchg(PAUSED, STOPPED)` succeeds immediately—no IPI needed; `@arch_state` is already valid in memory.

Slow Path (Live Preemption):

If `cb->state == RUNNING`, transitions `cmpxchg(RUNNING, STOPPING)` and sends a physical kick (`arch_cpu_preserved_kick` — x2APIC NMI / GIC SGI) to force VM-exit, flush hardware state to `@arch_state`, and publish STOPPED.

vCPUfd LUO Preservation

vCPU state & secondary page tables preservation via LUO

uAPI

- **KVM_CAP_VCPU_PRESERVE**
  Advertises vcpufd preservation capability
- The actual preservation of vmfd and vcpufd uses:

LIVEUPDATE_SESSION_PRESERVE_FD
LIVEUPDATE_SESSION_RETRIEVE_FD

- In-kernel RAM-only suspend and resume flow.
- Bypasses userspace round-trips via **`KVM_GET/SET_*`** ioctls.
- Enabled CoCo VM Suspend/Resume

KHO / ABI

include/linux/kho/abi/kvm.h

```
struct kvm_vcpu_ser {
  u32 vcpu_id;
  u64 vm_token;
  DECLARE_KHOSER_PTR(cb, struct kvm_caretaker_cb_ser *);
  DECLARE_KHOSER_PTR(arch_state, struct kvm_vcpu_arch_ser *);
};

struct kvm_luo_ser {
  u64 type;
  DECLARE_KHOSER_PTR(kho_folios, struct kvm_kho_folios_ser *);
};
```

vCPUfd LUO Preservation, cont.

x86 vCPU State Serialization

x86 ABI

- Reuses existing uAPI structs (`kvm_regs`, `sregs`, `events`, `lapic`)
- 64-byte aligned `xsave` enables direct in-place `XSAVE64 / XRSTOR64` in Caretaker mode.
- Variable-length `DECLARE_KHOSER_PTR` preserves dynamic `MSRs` & `CPUID` tables.
- kvm_vcpu_arch_ser, kvm_msrs, and kvm_cpuid2 are allocated together in one contiguous kho_alloc_preserve() buffer
- When Caretaker enabled it updates these in-place on each quantum / on attach

KHO / ABI

include/linux/kho/abi/kvm_x86.h

```
struct kvm_vcpu_arch_ser {
  struct kvm_regs regs;             /* KVM_GET/SET_REGS */
  struct kvm_sregs2 sregs;           /* KVM_GET/SET_SREGS2 */
  struct kvm_mp_state mp_state;     /* KVM_GET/SET_MP_STATE */
  struct kvm_xcrs xcrs;             /* KVM_GET/SET_XCRS */
  struct kvm_lapic_state lapic;     /* KVM_GET/SET_LAPIC */
  struct kvm_xsave xsave;           /* KVM_GET/SET_XSAVE */
  struct kvm_vcpu_events events;    /* KVM_GET/SET_VCPU_EVENTS */
  struct kvm_debugregs debugregs;   /* KVM_GET/SET_DEBUGREGS */
  struct kvm_clock_data clock;      /* KVM_GET/SET_CLOCK */
  /* KVM_GET/SET_MSRS */
  DECLARE_KHOSER_PTR(msrs, struct kvm_msrs *);
  /* KVM_GET/SET_CPUID2 */
  DECLARE_KHOSER_PTR(cpuid, struct kvm_cpuid2 *);
};
```

vCPUfd LUO Preservation, cont.

ARM64 vCPU State Serialization

ARM64 ABI

- Standard uAPI structures: `kvm_regs`, `mp_state`, `events`, and `kvm_vcpu_init`.
- Dynamic array `kvm_arm64_sysregs_ser` packs system & VGICv3 CPU interface registers in contiguous KHO memory.
- In-kernel accessors (`kvm_arm_sys_reg_read/write`) serialize state directly without VMM ioctl overhead.

KHO / ABI

include/linux/kho/abi/kvm_arm64.h

```
struct kvm_arm64_sysregs_ser {
  u32 num_sysregs;
  struct kvm_one_reg sysregs[];     /* KVM_GET/SET_ONE_REG */
};

struct kvm_vcpu_arch_ser {
  struct kvm_regs regs;             /* KVM_GET/SET_ONE_REG */
  struct kvm_mp_state mp_state;     /* KVM_GET/SET_MP_STATE */
  struct kvm_vcpu_events events;    /* KVM_GET/SET_VCPU_EVENTS */
  struct kvm_vcpu_init init;        /* KVM_ARM_VCPU_INIT */
  DECLARE_KHOSER_PTR(sysregs, struct kvm_arm64_sysregs_ser *);
};
```

Physical CPU LUO Preservation

Physical CPU preservation

uAPI

/sys/devices/system/cpu/cpu<N>/preserve

- Preserves CPU sysfs file descriptor into LUO session.
- Reuses CPU hotplug to migrate tasks, timers, and interrupts.
- Parks core in isolated `.cpu_preserved.*` sections with dedicated page tables.
- Survives kexec stop IPIs (offline) and incoming **smp_init()** (clears **cpu_present**).

KHO / ABI

include/linux/kho/abi/cpu.h

```
struct cpu_preserved_ser {
  u32 cpu;
  u32 state;
  u64 stack_pa;
  u64 hwid;
  DECLARE_KHOSER_PTR(session, struct cpu_preserved_session_ser *);
  DECLARE_KHOSER_PTR(next, struct cpu_preserved_ser *);
};
struct cpu_preserved_global_ser {
  u32 nr_cpu_words;
  u64 text_runtime_pa, text_runtime_size;
  u64 data_runtime_pa, data_runtime_size;
  DECLARE_KHOSER_PTR(cpus, struct cpu_preserved_ser *);
};
```

Physical CPU LUO Preservation, cont.

Address Space Preservation

IN-KERNEL API

Address Space & Paging

- **Isolated Address Mapping:** Configured via `cpu_preserved_as_ser`
- **Page Table Preservation:** Preserves dedicated page table roots (**`pgd_pa`**) across kexec reboots. And other job related data such as stack.

KHO / ABI

include/linux/kho/abi/cpu.h

```
struct cpu_preserved_as_ser {
  u64 pgd_pa;
  u64 nr_folios;
  DECLARE_KHOSER_PTR(folios, struct kho_block_header_ser * );
};
```

pCPU Page Table Walkers

Isolated Transitional Address Space Walkers

TRANSITION PAGE TABLES

Lean Isolated Mapping

The Problem:

Cloning the full linear direct map (GBs to TBs) into KHO wastes memory and adds security risk.

The Solution:

Build an isolated transitional page table mapping only what the preserved pCPU touches (<10 pages total).

Explicit Footprint:

- Sections: `.cpu_preserved.*, i.e. .text, .data, .ex_table, .bss`
- Dedicated preserved stacks & session metadata

ARCHITECTURE WALKERS

Arch Implementations

Kexec Trampoline Walkers:

x86: `kernel_ident_mapping_init()`

arm64: `trans_pgd_map_range()`

Allocator Hook (`cpu_preserved_as_alloc_page()`):

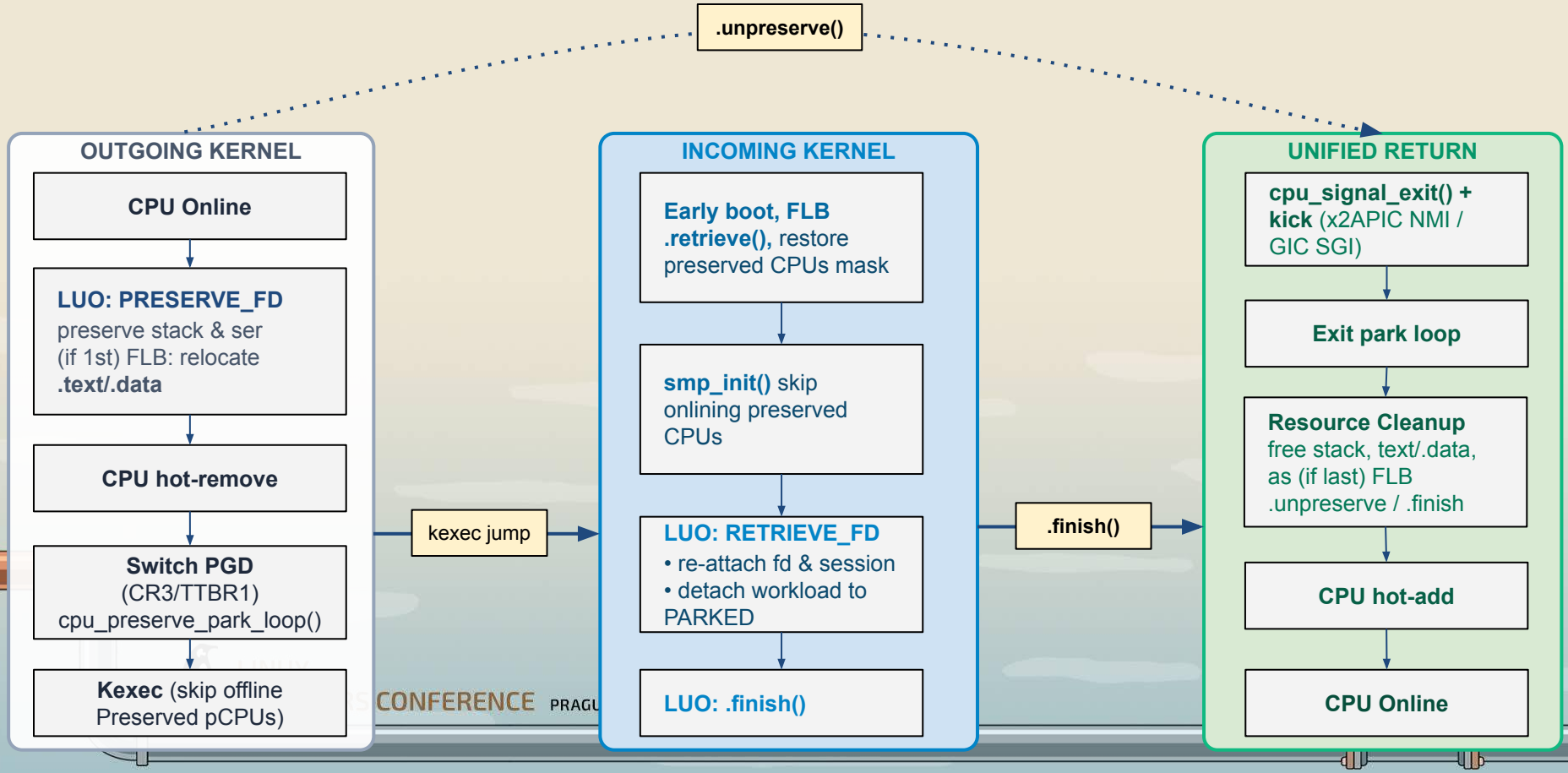
Allocates KHO-preserved page table pages and records them in `as_ser->pg_tables`.

Walk-Free Teardown:

Incoming kernel iterates `as_ser->pg_tables (kho_block)` to free all page table pages without arch teardown walkers.

Preserved pCPU Life-Cycle, FLB Handover & Shared Return Path

Layer 2 cpu_preserve • Outgoing Handover, Kexec Gap & Unified Return Path



On-Core Scheduler

Workload-Agnostic Session, Job Abstraction, Scheduler

KEY ARCHITECTURE

Session Binding (`oncore_session`)

• Bound 1:1 to cpu_preserved_session (sess->cps); attaches oncore_sched_cpu_worker() to preserved pCPUs via cpu_preserved_attach_workload().

• Handed over via `cpu_preserved_ser.oncore` → `oncore_session_ser`.

Workload-Agnostic Job Unit (`oncore_job`)

• Independent work unit (e.g., KVM Caretaker vCPU) registered via `oncore_session_submit_job()`.

KVM Caretaker

Handler/Hypervisor of Orphaned VMs (CONFIG_KVM_CARETAKER / virt/kvm/caretaker/)

KEY ARCHITECTURE

On-Core Job Wrapping (`kvm_caretaker_vcpu_pre/post_preserve`)

- Wraps preserved vCPU as `oncore_job` bound to `kvm_arch_vcpu_caretaker_run()`.
- Falls back to RAM-only preservation if LUO session has no pCPUs.

Execution Loop (`kvm_caretaker_vcpu_run`)

- Executes with IRQs disabled.
- Arms timer, loops guest entry/exit

Normalized Exit Classification

- **In-Loop Emulation:** IPIs, CPUID, MSRs → advances RIP & re-enters.
- **Yield:** HLT/PAUSE → `YIELD_IDLE`; Preempt → `QUANTUM_EXPIRED`.
- **Unhandled Stall:** MMIO/IO → records stall, returns `ONCORE_EXIT_STALL`.

KHO / ABI

include/linux/kho/abi/kvm.h

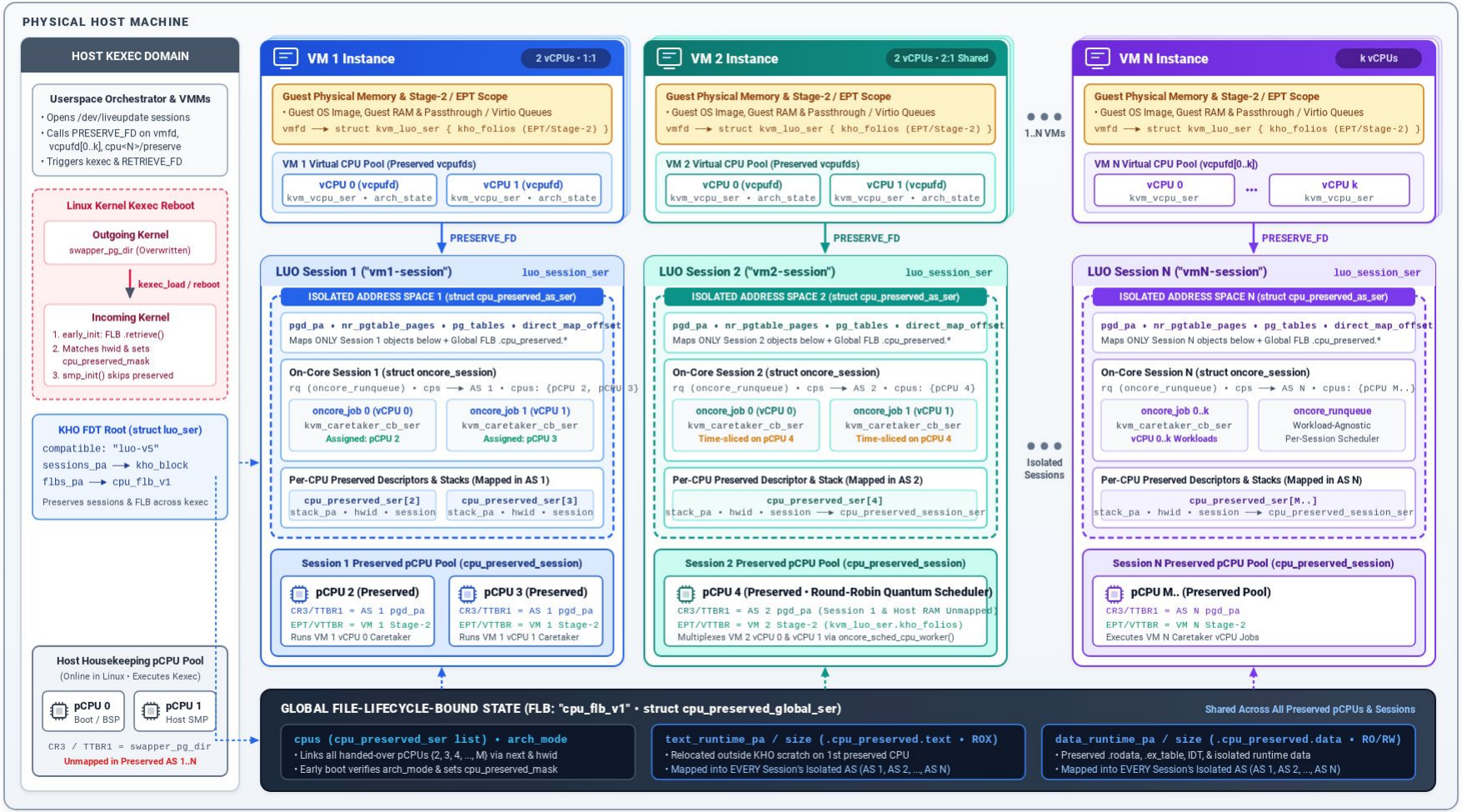
```
enum kvm_caretaker_state {
  KVM_CARETAKER_PAUSED = 0,  /* State in @arch_state */
  KVM_CARETAKER_RUNNING = 1, /* Live on preserved pCPU */
  KVM_CARETAKER_STOPPING = 2,/* Reclaim IPI */
  KVM_CARETAKER_STOPPED = 3, /* Ready to adopt */
  KVM_CARETAKER_STOPPED = 4, /* Unrecoverable fault */.
};

struct kvm_caretaker_cb_ser {
  u32 state, pcpu_id, vcpu_id;
  DECLARE_KHOSER_PTR(telemetry, struct
                     kvm_caretaker_telemetry_ser *);
};
```

OrphanVM Architecture

VMs, LUO Sessions, Per-Session Isolated Address Spaces (cpu_preserved_as_ser), On-Core Scheduler, Global FLB & Physical CPU Pools

Isolated Address Space Scope (cpu_preserved_as_ser)
Stage-2 / EPT Guest Memory Scope (kvm_luo_ser)
Global FLB Shared Runtime (.cpu_preserved.*)
Host Kexec Domain (Unmapped in AS)



source: https://raw.githubusercontent.com/soleen/orphanvm/a3f138d8d0baf944b93dfa90f290a985f27f9074/docs/ovm_host_architecture.svg

Discussion Topics & Open Questions

HYPERVISOR & VM-EXIT ARCHITECTURE

1. Zero-VM-Exit Baseline for Initial Landing?

- Land initial upstream with zero VM-exit emulation?
- Guest runs in HW virt; stalls (`ONCORE_EXIT_STALL`) on 1st exit until incoming KVM reclaims.

2. Sharing Entry/Exit & Fastpath Code with KVM

- **nvhe-style build:** Factor world-switch to operating context in Caretaker page; partially link (`ld -r`) into `.text.cpu_preserved` & prefix symbols to catch unpreserved calls.

3. vcpufd KHO Serialization ABI

- Reusing uAPI structs (`kvm_regs`, `kvm_sregs`) vs. fixed-width KHO structs (`struct kvm_vcpu_arch_ser`) paired with LUO ABI versioning.

DEVICES, HARDENING & EXTENSIONS

4. Direct Interrupt Injection Preservation

- Preserve IOMMU IRTEs & per-vCPU tables (`pi_desc`, `AVIC`, `GICv4.1`) pointed at preserved pCPU across kexec for running Orphaned VMs.

5. Paravirtual (virtio) Exits During kexec Gap

- Absorb posted `ioeventfd` doorbells in preserved memory; forward virtqueues to preserved service VM or SmartNIC DPU.

6. Nested Virt

- Preserve `kvm_nested_state` + shadow TDP; stall on nested world switches.

7. Kernel Caretaker

- Keep syscall-free userspace threads (DPDK, gVisor) running across kexec in preserved user page tables.

Resources & Community

PATCH SERIES & RESOURCES

Orphaned Virtual Machines

- Design Doc
- RFCv1 [RFC PATCH 00/47]
- Initial Proposal

Guest Memory & Base vmfd Preservation

- [PATCH v4 00/11] liveupdate: kvm: Guest_memfd preservation

Pass-Through Device & IOMMU Preservation

- PCI Core [PATCH v9 00/13]
- VFIO/PCI [PATCH v5 00/20]
- IOMMU [PATCH v5 00/18]

LUO & KHO Compatibility

- [RFC PATCH 0/3] Feature flags for ABI compatibility
- LPC'26 Session

COMMUNITY & COLLABORATION

Hypervisor Live Update Bi-Weekly Call

- **Schedule:** Mon 8:00 AM PDT (UTC-7) *(Next: Oct 19, 2026)*
- **Join Link:** https://meet.google.com/rjn-dmzu-hgq

Public Meeting Notes & Mailing Lists

- Lore Meeting Notes Archive

Active Workstreams Covered

- **Orphaned VMs:** kvm_luo, cpu_preserve, oncore & Caretaker
- **PCIe / VFIO / IOMMU:** Continuous DMA preservation across kexec
- **Guest Memory:** guest_memfd, memfd, HugeTLB, tmpfs & TDX
- **KHO Boot Memory:** bootmem, KHO XArray, PowerPC & LoongArch
- **Compatibility:** Cross-subsystem ABI versioning